CSC1046 OO Analysis & Design Submission 2 – Product Design

CSC1046 Scrabble Group 7

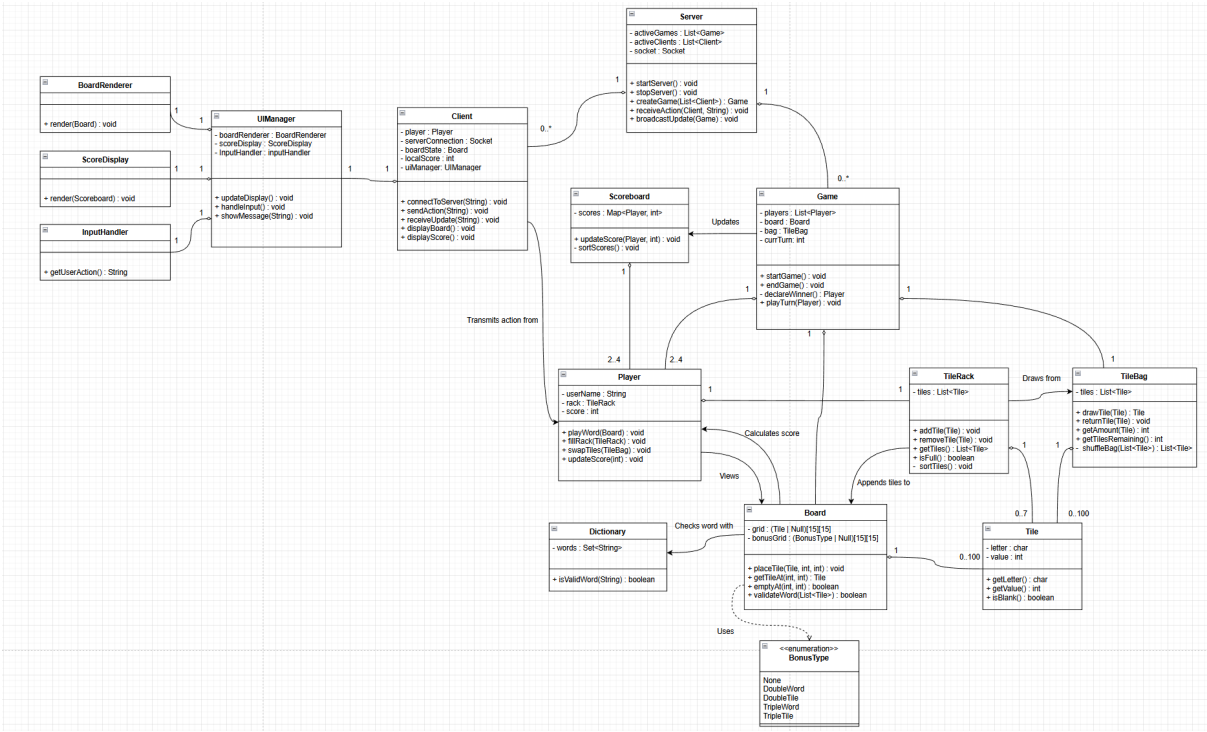
Table of Contents

0. Refined Use Case 3
1. Refined Class Diagram 4
1a. Changes 4
1b. Additions 4
2. Object Diagram 5
3. User Interface Mock-ups 6
3a. Network Implementation Idea 10
4. State Diagrams 11
4a. Game Flow 11
4b. Tile Placing 12
5. Sequence Diagrams 13
6.Collaboration Diagrams 16
7. Class Skeletons 18
7a. Core Classes 18
7b. Network Classes 25
7c. UI/Helper Classes 27
8. Meetings 29
9. PlantUML 34
10. Appendix 36

0. Refined Use Case

Use Case 1	**Start Game**	
Goal in Context	Initialise a new game with users, tiles, boards, racks and pre-determined turns	
Scope + Level	System, Core	
Preconditions	2-4 players have joined, dictionary source available	
Success End Condition	Users have joined the game and noted; Tile bag seeded and shuffled; board reset; racks assigned; first player determined.	
Failed End Condition	Fewer than 2 players; dictionary unavailable; initialization error.	
Primary Actor **Secondary Actors**	Players Game Engine	
Trigger	Players logs in and select “Start New Game”	
Description	**Step**	**Action**
	1.	Users Login/Register
	2.	Users Accept Invite to Game
	3.	Users Select Session Length
	4.	Game Engine creates/shuffles tile bag and resets board
	5.	Engine deals up to 7 tiles to each rack and chooses starting player
Extensions	**Step**	**Action**
		Cancel Game
Variations	**Step**	**Action**
		None

1. Refined Class Diagram



1a. Changes

BonusType

Added ‘NONE’ type to enum for easier premium cell validation.

TileBag

Added a getTilesRemaining() method for checking if allocating tiles is possible.

Added a shuffleBag() method for initalisation of bag.

Board

Board now checks word validity, accessing the Dictionary

1b. Additions

Added multiplayer functionality through the **Client and Server** classes.

Server hosts sessions for connected clients, broadcasting updates to them.

Client sends actions to the server, receiving updates in response.

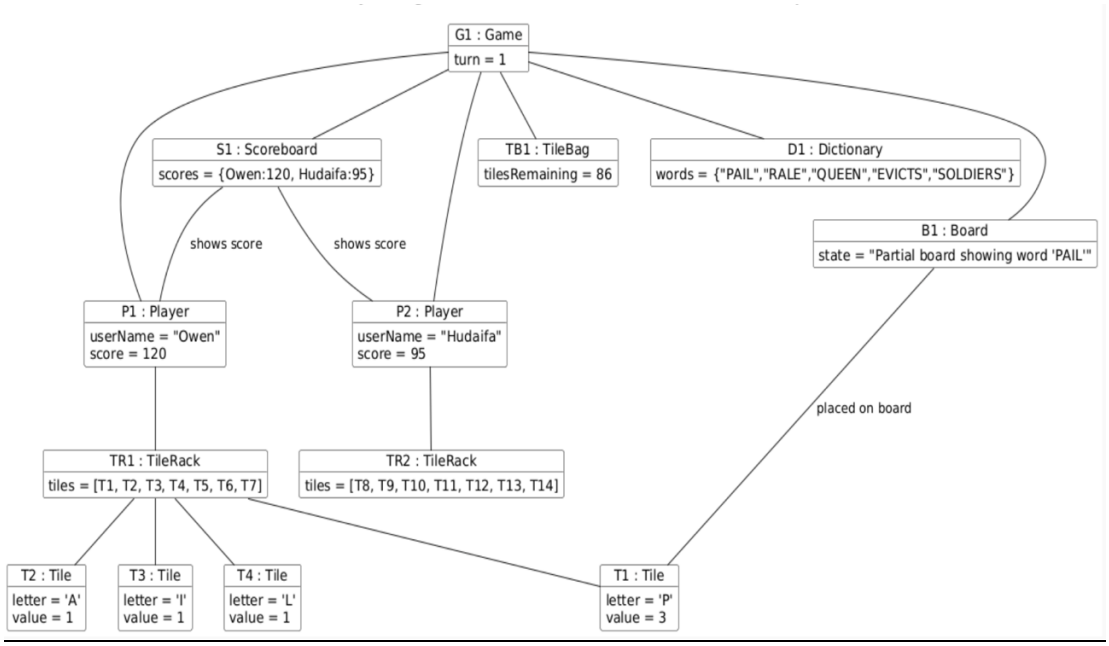
UIManager class added, which manages other UI/helper classes.

Helper/UI Classes: **BoardRenderer**, **ScoreDisplay** and **InputHandler**

The client uses this manager to display game state updates to the user.

InputHandler allows the client to send actions to the server

2. Object Diagram

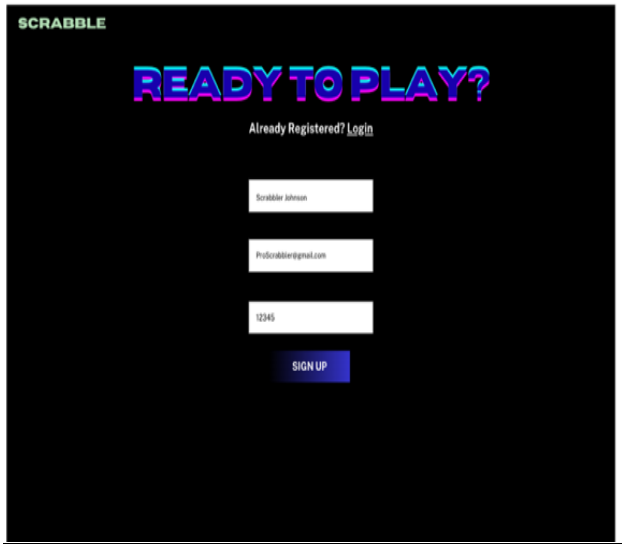


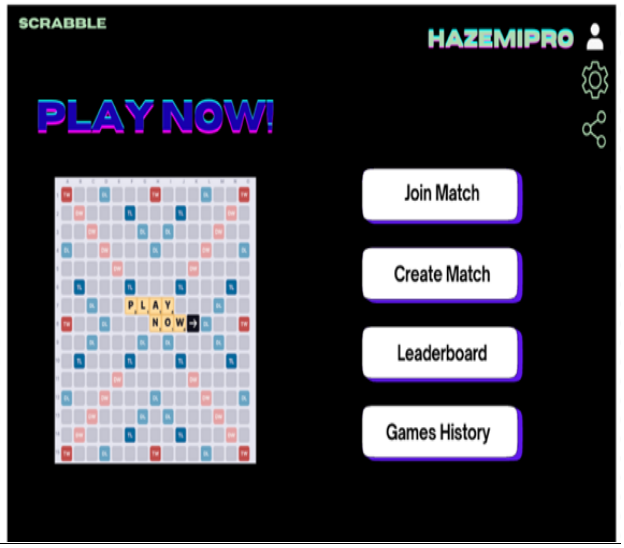
3. User Interface Mock-ups

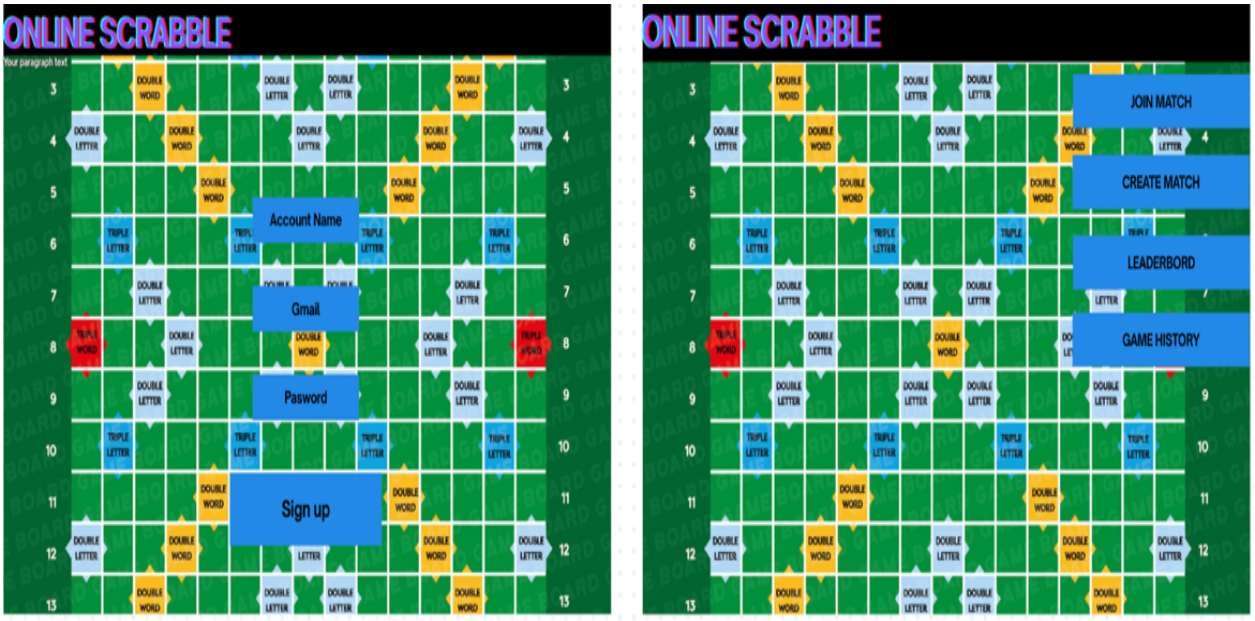
In our first stage of designing we took inspiration from the current variations of scrabble. We reimagined them in 3 different variations. Our first design was a simplistic black one, loosely following an arcade colour scheme design. However, we felt that it could cause eye strain from looking at it for too long due to the colour contrasts. Our second design used the original board games as the backdrop for the menu. However the main menu felt cluttered and made it difficult for users to find the buttons, the design did not follow sharpening/prioritising principle of UI.

We took on this opinion and decided, that we shall keep the structure but alter the colour scheme. Furthermore, we carried out a vote and decided that we should follow a light blue with a retro style font for the UI. This has turned out to become a success based on our Final Design mock-up as we as a team found the user experience to being more fluent, modern.

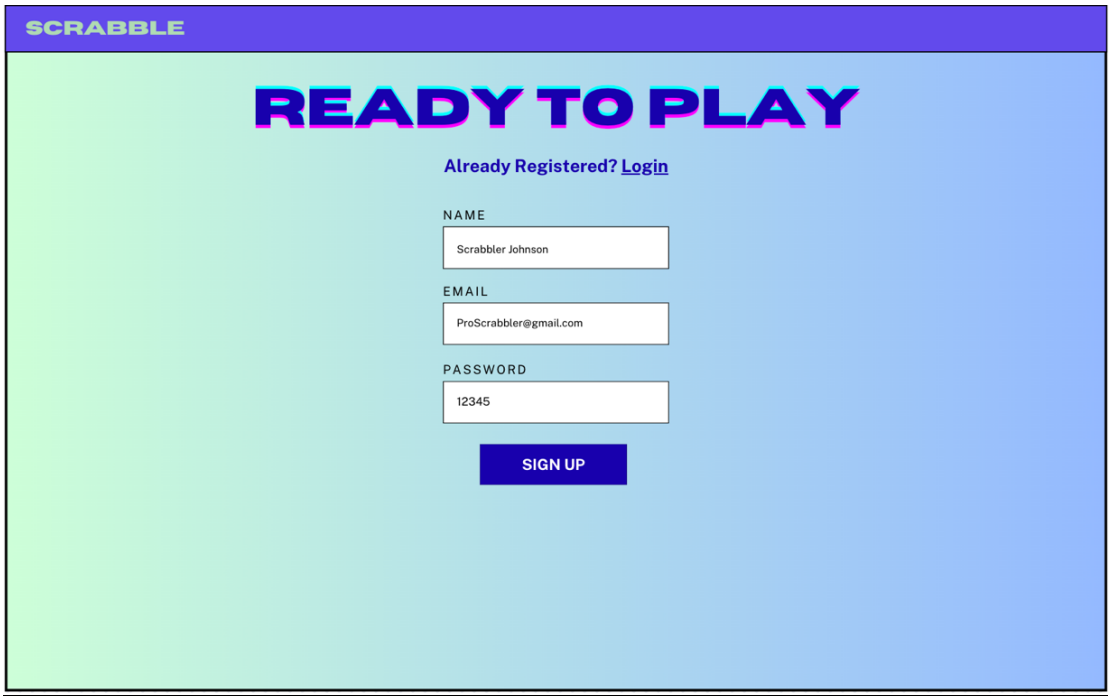
Initial Designs







Final Design Mock-Up



SCRABBLE HAZEMIPRO

PLAY NOW!



Join Match

Create Match

Leaderboard

Games History

SCRABBLE HAZEMIPRO

MATCH CREATION



Players

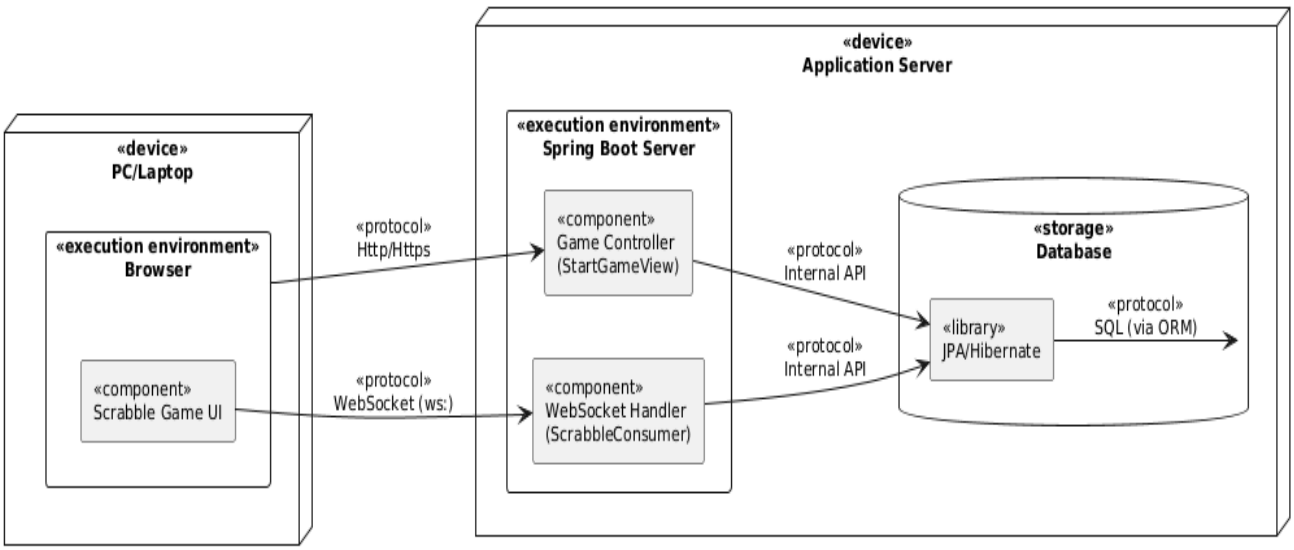
Bag Size

Length

Start


SCRABBLE
HAZEMIPRO
Quit
PLAYERS
Hazem - 55
6:43
Ryan - 56
5:22
Philip - 85
4:44
Owen - 68
6:32
TILE BAG
Tiles Left: 100
Consonants: 64
Vowels: 36
A B C D E F G H I J
9 2 2 4 12 2 3 2 5 9
K L M N O P Q R S T
1 1 4 6 8 2 0 6 4 6
U V W X Y Z
4 2 2 1 2 1 2
S A E T R L N
Swap
Submit
Skip

3a. Network Implementation Idea




```
// 1. WebSocket Setup (Client-side JavaScript):

let socket = new WebSocket('ws://your-server-url/ws/scrabble/');
socket.onmessage = function(e) {
    let data = JSON.parse(e.data);
    handleGameUpdate(data);
};
function sendMove(move) {
    socket.send(JSON.stringify({'type': 'move', 'data': move}));
}

// 2. WebSocket Handler (Server-side Java with Spring):

@ServerEndpoint("/ws/scrabble/{gameId}")
public class ScrabbleConsumer {
    @OnOpen
    public void connect(Session session, @PathParam("gameId") String gameId) {
        session.getUserProperties().put("game_id", gameId);
        self.accept();
    }

    @OnMessage
    public void receive(Session session, String textData) {
        JSONObject data = new JSONObject(textData);
        if (data.getString("type").equals("move")) {
            updateGameState(data.getJSONObject("data"));
        }
    }

    public void updateGameState(JSONObject move) {
        Game game = Game.objects.get(id=self.game_id);
        game.applyMove(move);
        game.save();
        async_to_sync(self.channel_layer.group_send)(
            "game_" + self.game_id,
            {"type": "game_update", "message": {"move": move, "state": game.getState()}}
        );
    }
}
```

```
// 3. Database Models (Spring JPA):

@Entity
class Player {
    @Id @GeneratedValue private Long id;
    private String name;
    @ManyToOne private User user;
}

@Entity
class Game {
    @Id @GeneratedValue private Long id;
    @OneToMany private List<Player> players;
    @Column(columnDefinition = "TEXT") private String currentState;

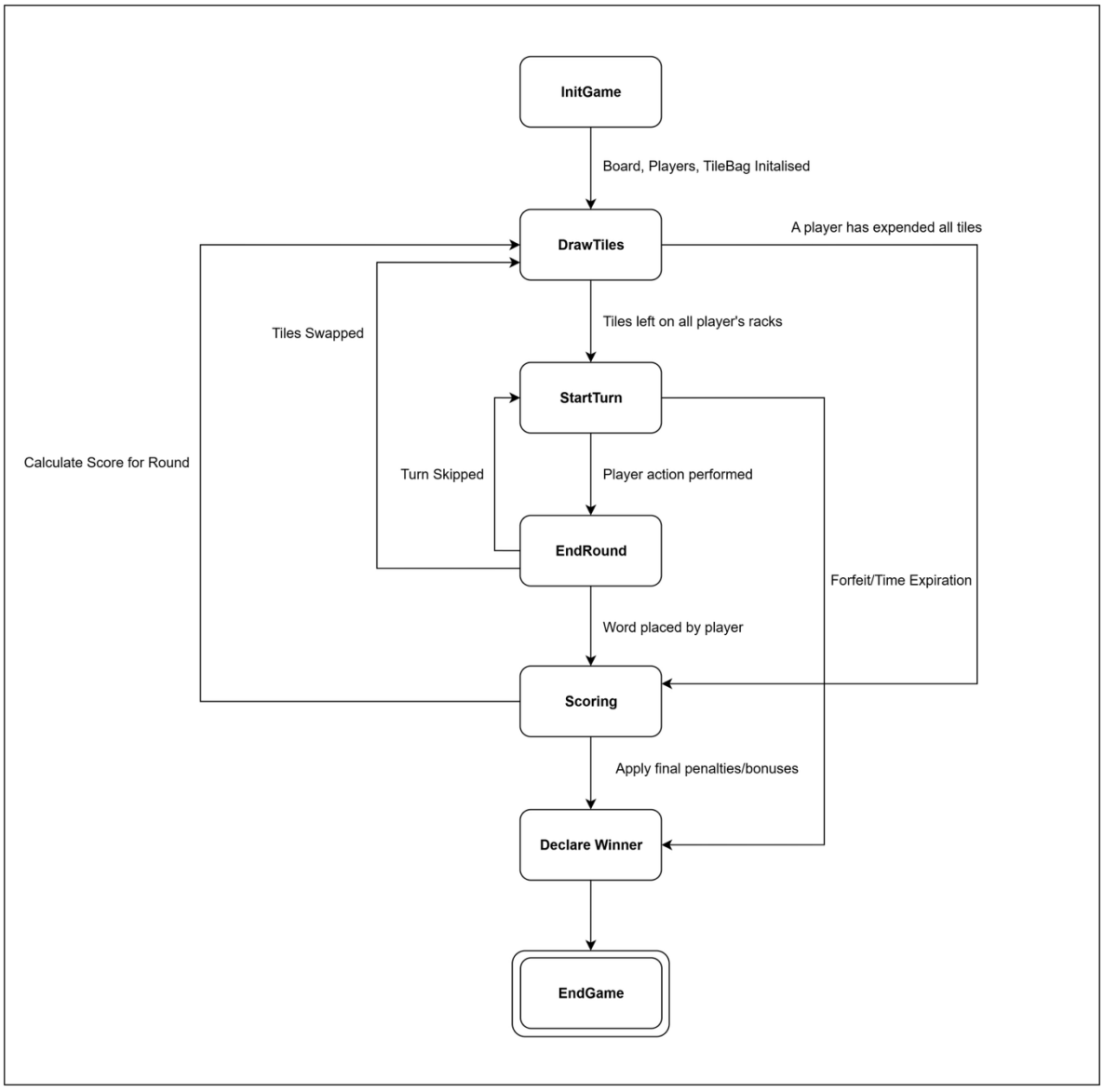
    public void applyMove(JSONObject move) {
        // Logic to apply move and update current_state
        pass
    }
    public JSONObject getState() {
        return new JSONObject(self.currentState);
    }
}

// 4. HTTP API (Spring Controller):
@RestController
class StartGameView {
    @PostMapping("/api/start-game")
    public Map<String, Object> post(@RequestBody Map<String, Object> request) {
        players = [Player.objects.get(id=id) for id in request.POST.getlist('player_ids')];
        game = Game.objects.create();
        game.players.set(players);
        game.save();
        return JsonResponse({"game_id": game.id});
    }
}

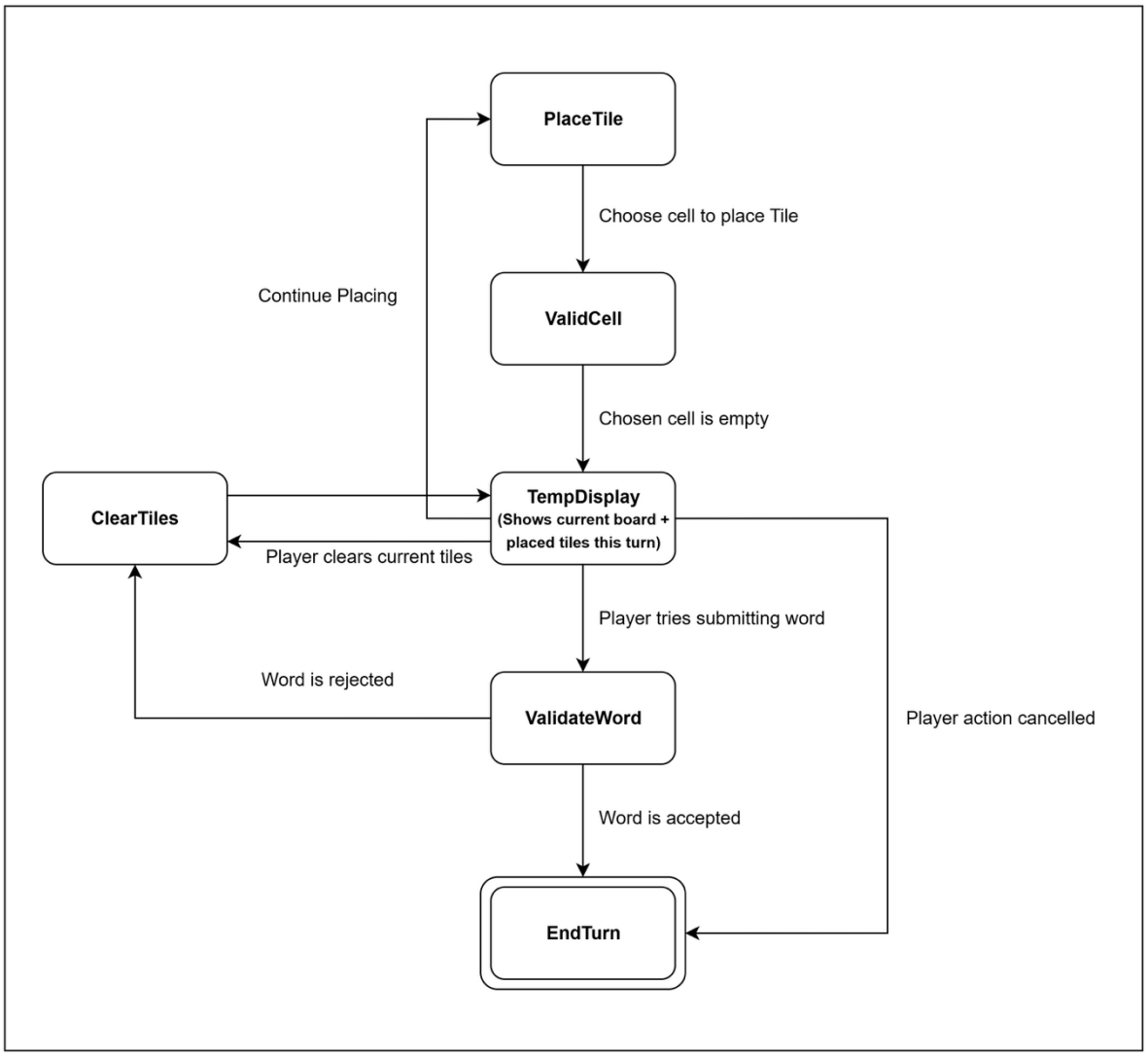
// 5. URL Configuration (urls.java):
urlpatterns = [
    path('api/start-game/', StartGameView.as_view(), name='start_game'),
    path('ws/scrabble/<int:game_id>/', ScrabbleConsumer.as_asgi()),
]
```

4. State Diagrams

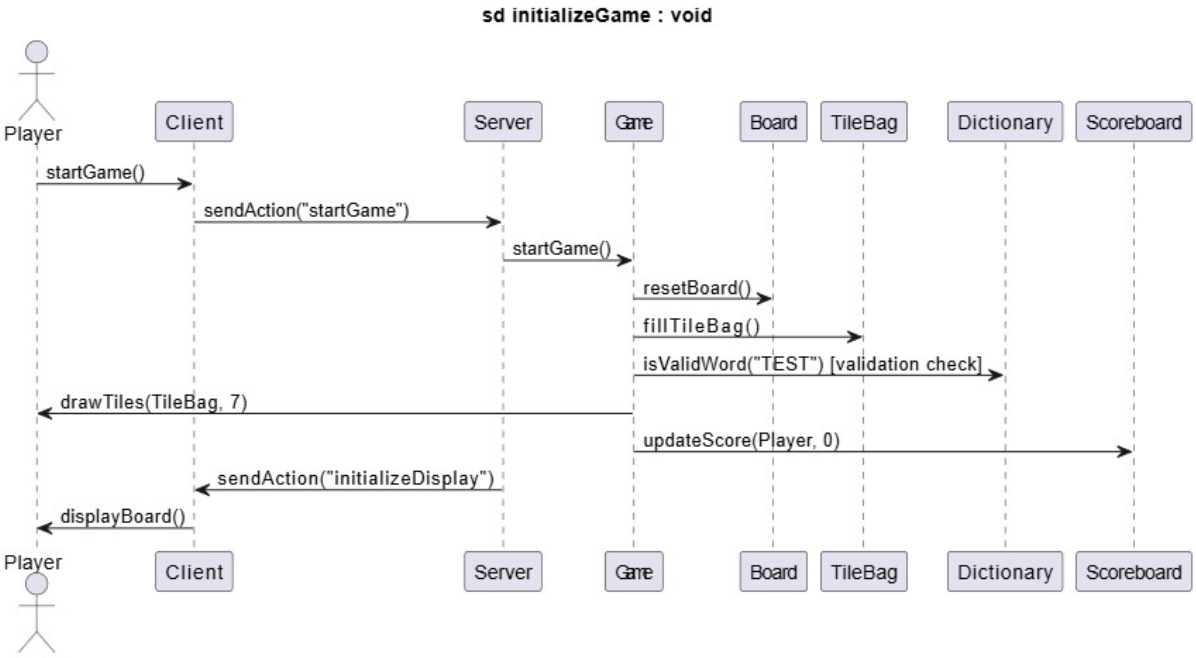
4a. Game Flow

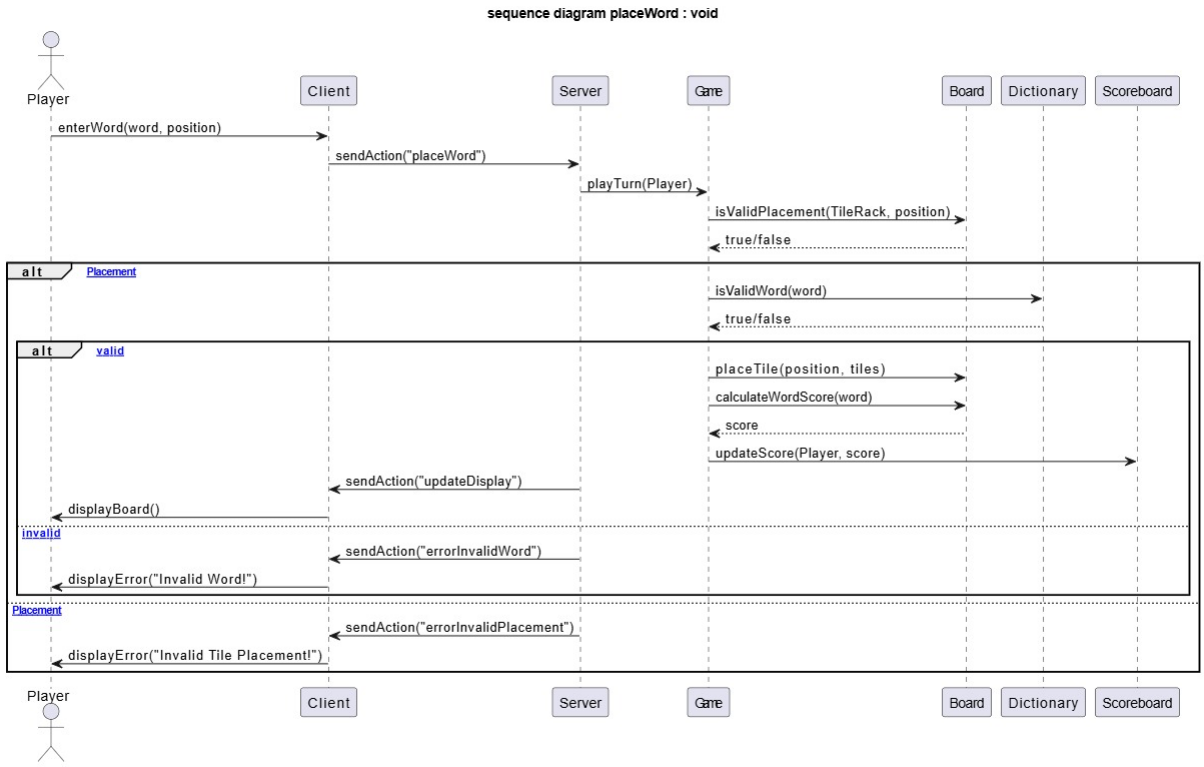


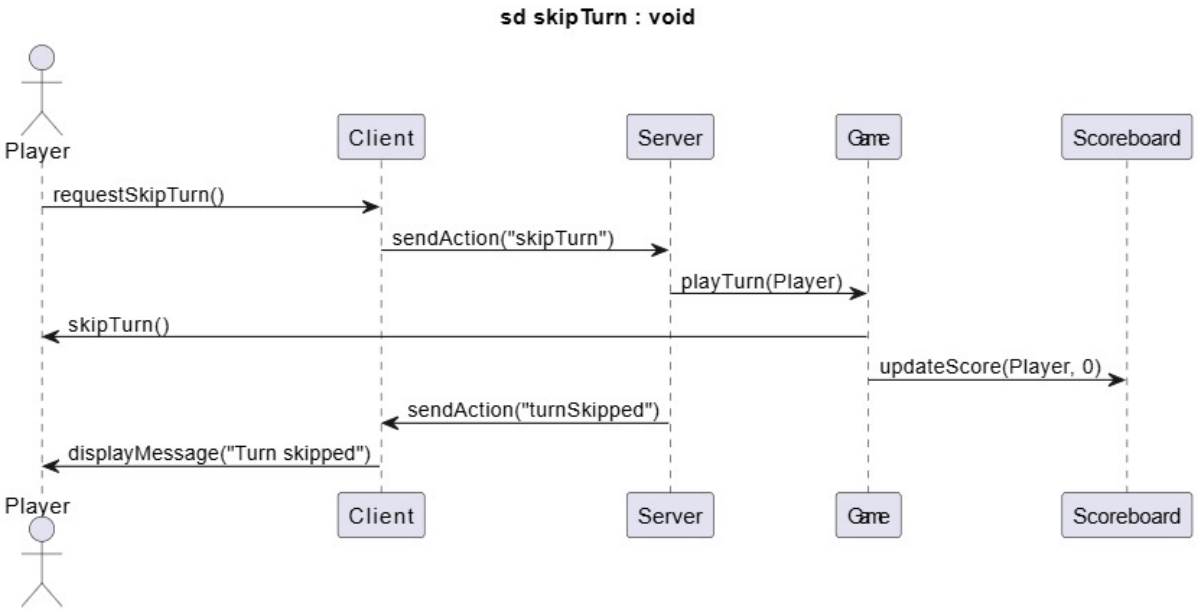
4b. Tile Placing

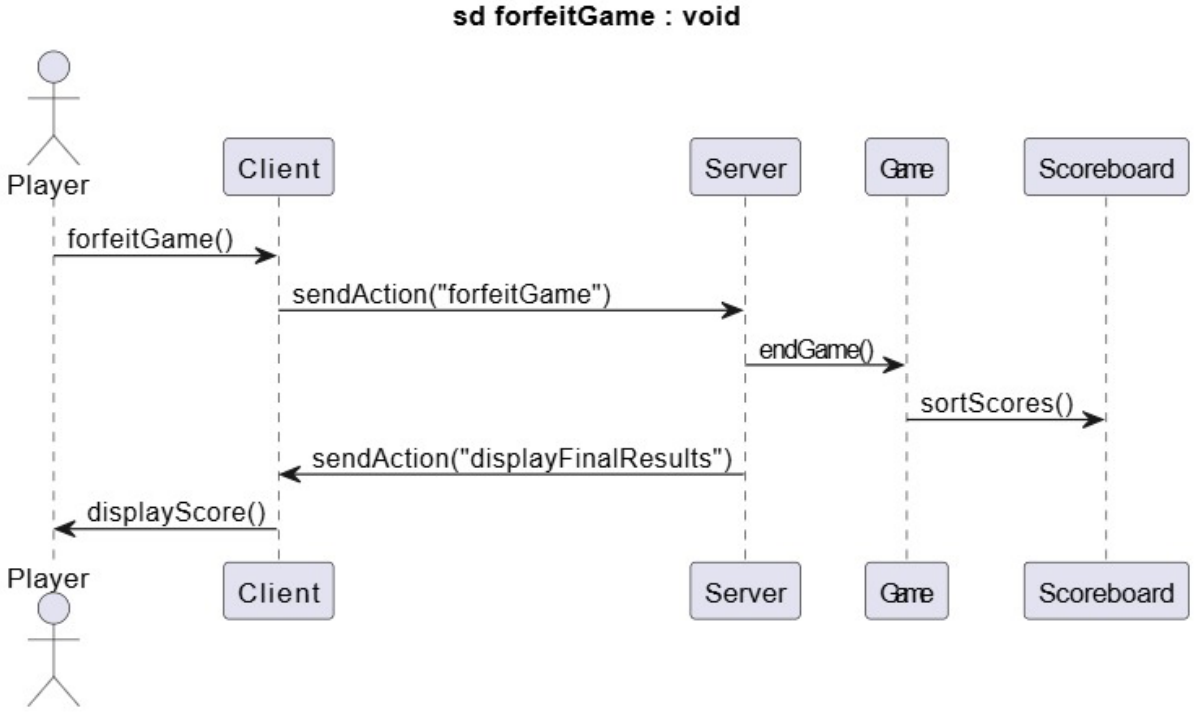


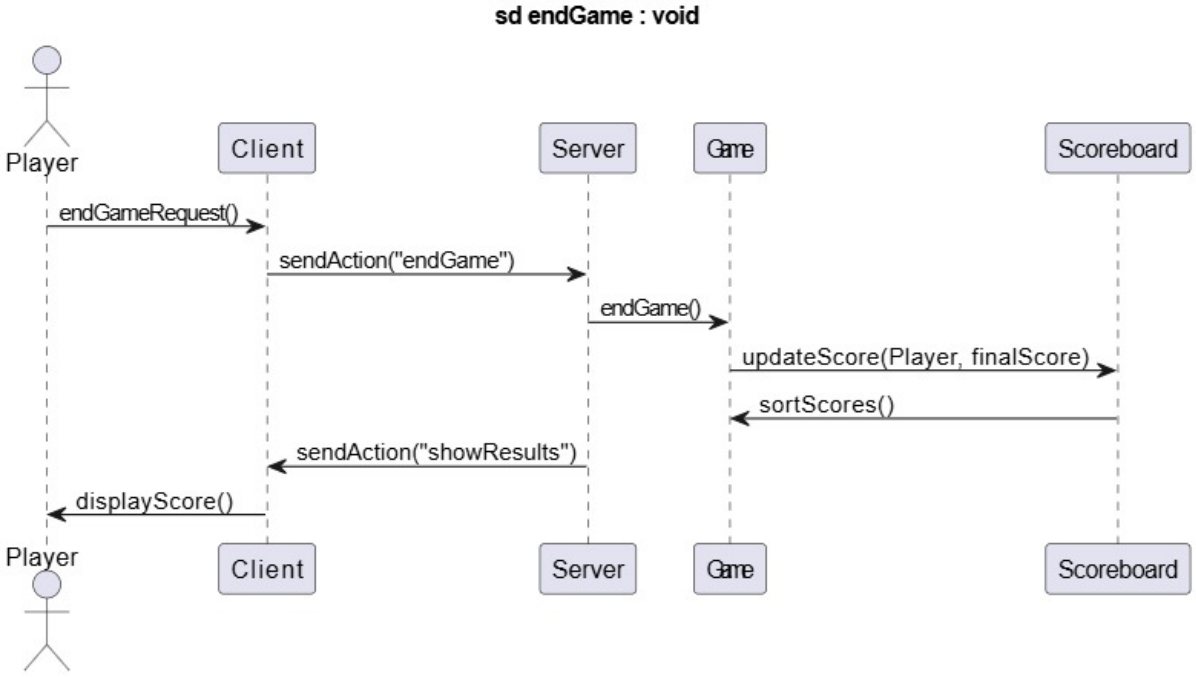
5. Sequence Diagrams





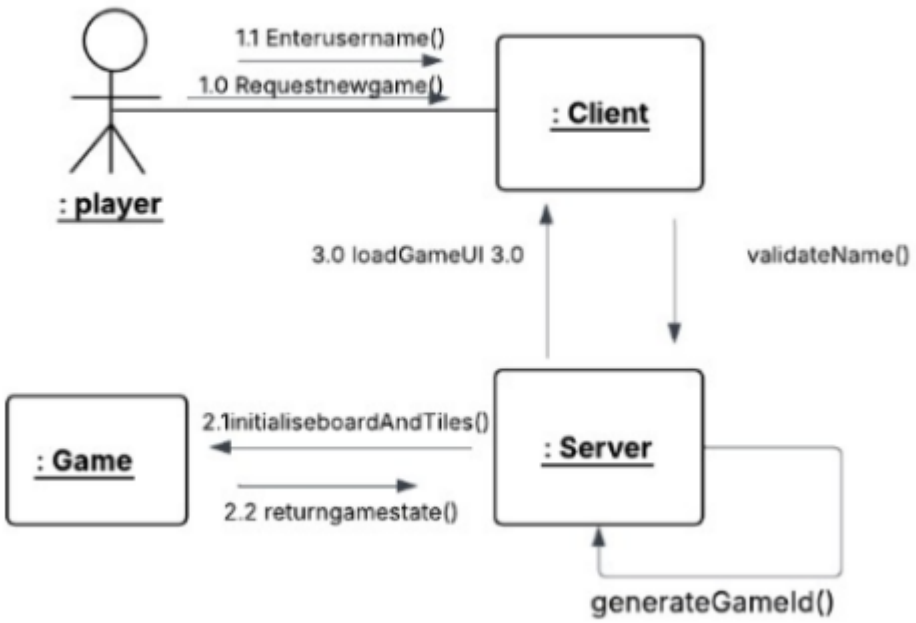

sd skipTurn : void
Player
Client
Server
Game
Scoreboard
requestSkipTurn()
sendAction("skipTurn")
playTurn(Player)
skipTurn()
updateScore(Player, 0)
sendAction("turnSkipped")
displayMessage("Turn skipped")
Player
Client
Server
Game
Scoreboard


sd forfeitGame : void
Player
Client
Server
Game
Scoreboard
forfeitGame()
sendAction("forfeitGame")
endGame()
sortScores()
sendAction("displayFinalResults")
displayScore()
Player
Client
Server
Game
Scoreboard

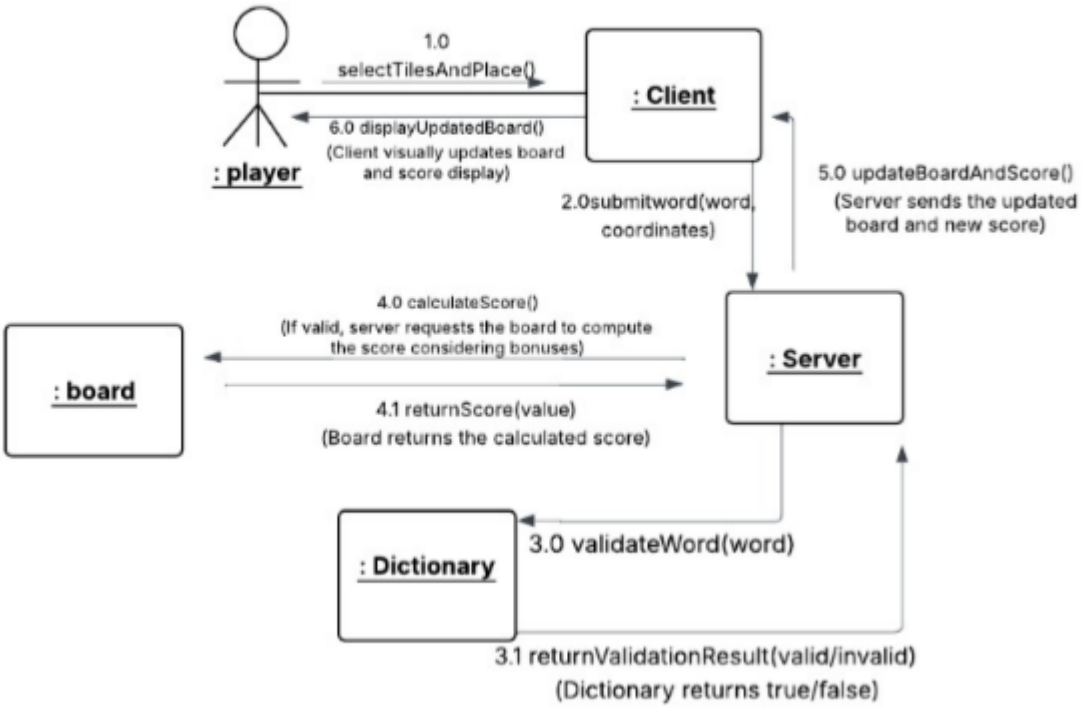

sd endGame : void
Player
Client
Server
Game
Scoreboard
endGameRequest()
sendAction("endGame")
endGame()
updateScore(Player, finalScore)
sortScores()
sendAction("showResults")
displayScore()
Player
Client
Server
Game
Scoreboard

6.Collaboration Diagrams

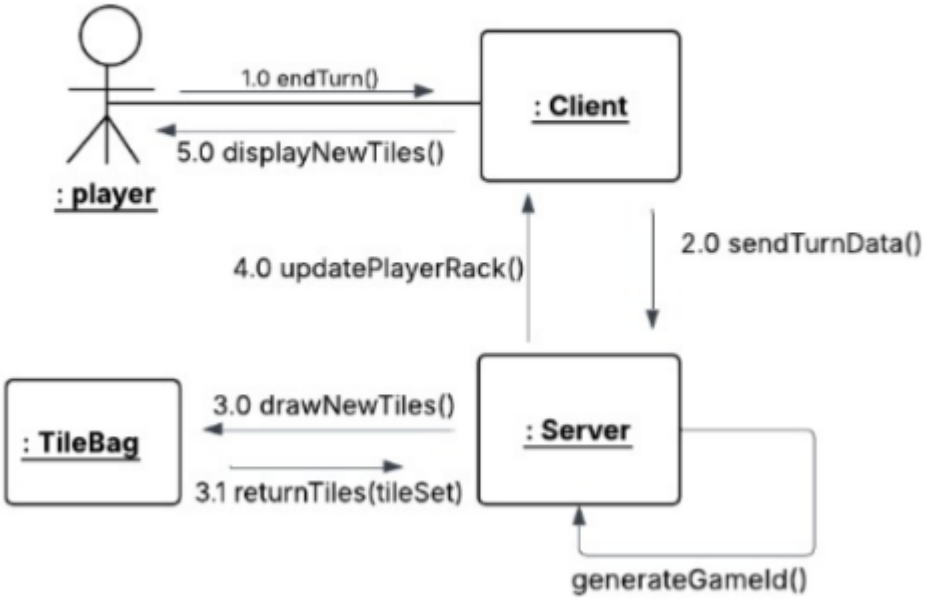
1. Initialise Game



2. place Word



3. End game


1.0 endTurn()
: Client
5.0 displayNewTiles()
: player
2.0 sendTurnData()
4.0 updatePlayerRack()
: TileBag
3.0 drawNewTiles()
: Server
3.1 returnTiles(tileSet)
generateGameId()

7. Class Skeletons

7a. Core Classes

Tile

```java
/**
 * Represents a tile used in Scrabble.
 * A tile contains a 'letter' (alphabet or blank) and a point value.
 */
public class Tile
{
    private char letter;
    private int value;

    public Tile(char letter, int value)
    {
        this.letter = letter;
        this.value = value;
    }

    public char getLetter() { return letter; }

    public int getValue() { return value; }

    public boolean isBlank() { return letter == ' '; }
}
```

TileRack

```java
/**
 * Represents the tiles a player has access to currently.
 * Responsible for managing and modifiying these tiles.
 */
public class TileRack
{
    private List<Tile> tiles;

    public TileRack()
    {
        this.tiles = new ArrayList<>();
    }

    public void addTile(Tile tile) {}

    public void removeTile(Tile tile) {}

    public List<Tile> getTiles() { return tiles; }

    public boolean isFull() { return tiles.size() >= 7; }

    private void sortTiles() {}
}
```

TileBag

```java
/**
 * Represents a bag containing all unused tiles in the game.
 * Responsible for drawing, exchanging, and counting tiles.
 */
public class TileBag
{
    private List<Tile> tiles;

    public TileBag()
    {
        this.tiles = new ArrayList<>();
    }

    public Tile drawTile() {return null; }

    public void returnTile(Tile tile) {}

    public boolean getAmount(Tile) { return tiles.isEmpty(); }

    public int getTilesRemaining() { return tiles.size(); }

    private List<Tile> shuffleBag(tiles) {return null;}
}
```

BonusType

```
/**
 * Enum representing all premium cells that can appear on the board.
 */
enum BonusType
{
    NONE,
    DOUBLE_LETTER,
    TRIPLE_LETTER,
    DOUBLE_WORD,
    TRIPLE_WORD
}
```

Board

```
/**
 * Represents the Scrabble board.
 * Each cell is either empty or contains a tile.
 * Some cells are premium cells.
 */
public class Board
{
    private static final int SIZE = 15; // declaration to remove magic number
    private Tile[][] grid;
    private BonusType[][] bonusGrid;

    public Board()
    {
        grid = new Tile[SIZE][SIZE];
        bonusGrid = new BonusType[SIZE][SIZE];
    }

    public boolean placeTile(Tile tile, int row, int col) { return false; }

    public Tile getTileAt(int row, int col) { return grid[row][col]; }

    public boolean isEmptyAt(int row, int col) { return grid[row][col] == null; }

    public boolean validateWord(List<Tile> tiles) { return false; }
}
```

Player

```java
/**
 * Represents a player.
 * Holds the player's username, tiles, and score.
 * Takes user action for their turn.
 */
public class Player
{
    private String userName;
    private TileRack rack;
    private int score;

    public Player(String userName)
    {
        this.userName = userName;
        this.rack = new TileRack();
        this.score = 0;
    }

    public String getUsername() { return userName; }

    public TileRack getRack() { return rack; }

    public int getScore() { return score; }

    public void playWord(Board board) {}

    public void swapTiles(TileBag bag) {}

    public void updateScore(int points) { this.score += points; }

    public void fillRack(TileRack rack) {}
}
```

Dictionary

```java
/**
 * Represents a dictionary of valid Scrabble words.
 * Used by the board to verify words.
 */
public class Dictionary
{
    private Set<String> words;

    public Dictionary(Set<String> words)
    {
        this.words = words;
    }

    public boolean isValidWord(String word) { return false; }
}
```

Scoreboard

```java
/**
 * Tracks all players' scores.
 * Responsible for score updating and sorting.
 */
public class Scoreboard
{
    private Map<Player, Integer> scores;

    public Scoreboard()
    {
        this.scores = new HashMap<>();
    }

    public void updateScore(Player player, int points) {}

    public void displayScores() {}

    /** Sorts the scoreboard by score in descending order. */
    public void sortScores() {}
}
```

Game

```java
/**
 * Manages the main flow of the game.
 * Coordinates turn order and scoring for players.
 * Declares the winner of the game.
 */
public class Game
{
    private List<Player> players;
    private Board board;
    private TileBag bag;
    private int currTurn;

    public Game(List<Player> players)
    {
        this.players = players;
        this.dictionary = new Dictionary();
        this.board = new Board();
        this.bag = new TileBag();
        this.currTurn = 0;
    }

    public void startGame() {}

    public void playTurn(Player player) {}

    public void endGame() {}

    public Player declareWinner() {return null;}
}
```

7b. Network Classes

Client

```
/**
 * Represents a connected client in a session.
 * Manages communication between a player and the server.
 */
class Client
{
    private Player player;
    private Socket serverConnection;
    private Board boardState;
    private int localScore;
    private UIManager uiManager;

    public Client(Player player) {}

    /** Connects this client to a remote server. */
    public void connectToServer(String address) {}

    /** Sends an action (move, swap, pass, etc) TO the server. */
    public void sendAction(String action) {}

    /** Receives updates (board, scores, etc.) FROM the server. */
    public void receiveUpdate(String update) {}

    // Methods for managing game display, using the UI Manager
    public void displayBoard() {}
    public void displayScore() {}
}
```

Server

```
/**
 * Represents the game server.
 * Manages all active games and clients connected to them.
 */
class Server
{
    private List<Game> activeGames;
    private List<Client> activeClients;
    private Socket socket;

    public Server() {}

    public void startServer() {}

    public void stopServer() {}

    /** Creates a new game session for connected clients. */
    public Game createGame(List<Client> clients) { return null; }

    /** Receives action (move, swap, pass, etc.) FROM a client. */
    public void receiveAction(Client client, String action) {}

    /** Broadcasts game updates (board, scores, etc.) TO ALL clients. */
    public void broadcastUpdate(Game game) {}
}
```

7c. UI/Helper Classes

UI Manager

```
/**
 * Manages the UI components for a client.
 * This includes rendering, input handling, and message display.
 */
class UIManager
{
    private BoardRenderer boardRenderer;
    private ScoreDisplay scoreDisplay;
    private InputHandler inputHandler;

    public UIManager() {}

    /** Updates all UI elements (board, score, etc.). */
    public void updateDisplay() {}

    /** Handles user input (e.g playing or swapping tiles). */
    public void handleInput() {}

    /** Displays a message to a user. */
    public void showMessage(String message) {}
}
```

BoardRenderer

```
/**
 * Handles the rendering of the game board for the user interface.
 */
class BoardRenderer
{
    public BoardRenderer() {}
    public void render(Board board) {}
}
```

ScoreDisplay

```
/**
 * Handles the rendering of the scoreboard for the user interface.
 */
class ScoreDisplay
{
    public ScoreDisplay() {}

    public void render(Scoreboard scoreboard) {}
}
```

InputHandler

```
/**
 * Handles all user input.
 */
class InputHandler
{
    public InputHandler() {}

    /** Gets an action (command) from the user. */
    public String getUserAction() { return ""; }
}
```

8. Meetings

Monday 29/09 - 16:00 - 16:30

Location / Platform: L.101

Attendees:

- The purpose of the meeting was to assign each member their part of the work and make sure responsibilities were clear for Product Design.
- We discussed and divided tasks so everyone knew exactly which section they would be handling.
- After confirming the division of work, we reviewed and explained how the UI of the project is going to look.
- Each member shared ideas on the layout, design, and flow to ensure consistency across the project.
- By the end of the meeting, everyone had a clear role and a shared vision of the user interface.

Thursday 02/10 - 1PM-1:30PM

Location / Platform: L.101

Attendees:

- The meeting was a follow-up from the previous session where everyone was asked to create a simple sample of their ongoing work.
- All members attended and presented their progress, showing small but meaningful updates.
- Each participant shared what they had accomplished so far, along with any challenges or obstacles they faced.
- The group offered feedback, advice, and practical tips to help each other improve and move forward.
- Discussion focused on how to develop the initial samples into more refined drafts for the next meeting.
- The goal for the next session is for everyone to bring a more structured and detailed version of their work, not a final product but a clearer draft.
- The team agreed to continue supporting one another through feedback and collaboration as the projects progress.

Friday 10/10 - 6PM-6:30PM

Location / Platform: Discord Meeting

Attendees:

- The purpose of this meeting was to review our current progress. We noted that there is still plenty of time remaining before the project deadline, allowing us to plan carefully for the next stages.
- We discussed the UI design mock-ups and decided to delay voting until everyone has had time to review the latest updates. After further discussion, we selected a preferred design and agreed to move forward with it.
- The team will meet again next week to review progress and discuss any new updates.

Friday 17/10 - 21:00 - 21:30

Location / Platform: Discord Meeting

Attendees:

- The purpose of this meeting was to review the team's current progress and discuss any recent updates.
- We reviewed the existing work and identified a few changes needed to ensure that all names remain consistent across every diagram.
- The team agreed to apply these naming adjustments and other updates before the next meeting.
- We plan to meet again next week after the upcoming lecture, which will explain how to submit the project correctly and ensure this part is completed properly.

Friday 22/10 - 18:00 - 19:30

Location / Platform: WhatsApp Meeting

Attendees:

• The purpose of this meeting was to finalize the prototype design and ensure that all components align with the project requirements.

• We reviewed the updated wireframes and discussed feedback from the previous consultation session.

• The team decided to refine the colour scheme and adjust the button spacing for better visual consistency.

Friday 02/11 - 18:00 - 19:30

Location / Platform: WhatsApp Meeting

Attendees:

• The purpose of this meeting was to proofread the document.

• We reviewed the updated sections in accordance with the rubric.

• The team decided to submit the task and started planning for the next submission.

9. PlantUML

```
@startuml
title sd initializeGame : void

actor "Player" as Player
participant "Client" as Client
participant "Server" as Server
participant "Game" as Game
participant "Board" as Board
participant "TileBag" as TileBag
participant "Dictionary" as Dictionary
participant "Scoreboard" as Scoreboard

Player -> Client: startGame()
Client -> Server: sendAction("startGame")
Server -> Game: startGame()
Game -> Board: resetBoard()
Game -> TileBag: fillTileBag()
Game -> Dictionary: isValidWord("TEST") [validation check]
Game -> Player: drawTiles(TileBag, 7)
Game -> Scoreboard: updateScore(Player, 0)
Server -> Client: sendAction("initializeDisplay")
Client -> Player: displayBoard()
@enduml
```

```
@startuml
title sd placeWord : void

actor "Player" as Player
participant "Client" as Client
participant "Server" as Server
participant "Game" as Game
participant "Board" as Board
participant "Dictionary" as Dictionary
participant "Scoreboard" as Scoreboard

Player -> Client: enterWord(word, position)
Client -> Server: sendAction("placeWord")
Server -> Game: playTurn(Player)
Game -> Board: isValidPlacement(TileRack, position)
Board --> Game: true/false

alt [Valid Placement]
    Game -> Dictionary: isValidWord(word)
    Dictionary --> Game: true/false
    alt [Word valid]
        Game -> Board: placeTile(position, tiles)
        Game -> Board: calculateWordScore(word)
        Board --> Game: score
        Game -> Scoreboard: updateScore(Player, score)
        Server -> Client: sendAction("updateDisplay")
        Client -> Player: displayBoard()
    else [Word invalid]
        Server -> Client: sendAction("errorInvalidWord")
        Client -> Player: displayError("Invalid Word!")
    end
else [Invalid Placement]
    Server -> Client: sendAction("errorInvalidPlacement")
    Client -> Player: displayError("Invalid Tile Placement!")
end
@enduml

```

```
@startuml
title sd skipTurn : void

actor "Player" as Player
participant "Client" as Client
participant "Server" as Server
participant "Game" as Game
participant "Scoreboard" as Scoreboard

Player -> Client: requestSkipTurn()
Client -> Server: sendAction("skipTurn")
Server -> Game: playTurn(Player)
Game -> Player: skipTurn()
Game -> Scoreboard: updateScore(Player, 0)
Server -> Client: sendAction("turnSkipped")
Client -> Player: displayMessage("Turn skipped")
@enduml
```

```
@startuml
title sd forfeitGame : void

actor "Player" as Player
participant "Client" as Client
participant "Server" as Server
participant "Game" as Game
participant "Scoreboard" as Scoreboard

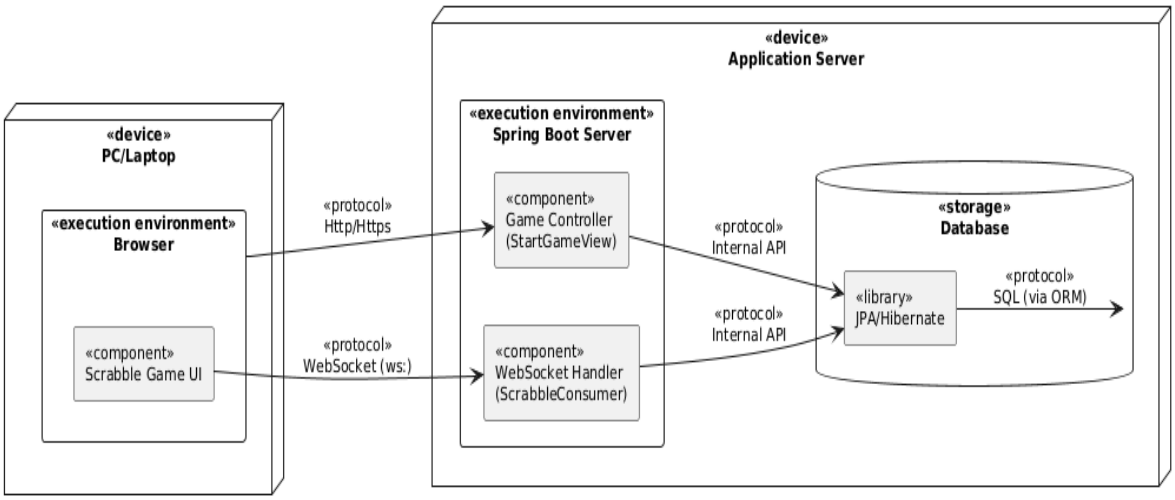
Player -> Client: forfeitGame()
Client -> Server: sendAction("forfeitGame")
Server -> Game: endGame()
Game -> Scoreboard: sortScores()
Server -> Client: sendAction("displayFinalResults")
Client -> Player: displayScore()
@enduml
```

```
@startuml
title sd endGame : void

actor "Player" as Player
participant "Client" as Client
participant "Server" as Server
participant "Game" as Game
participant "Scoreboard" as Scoreboard

Player -> Client: endGameRequest()
Client -> Server: sendAction("endGame")
Server -> Game: endGame()
Game -> Scoreboard: updateScore(Player, finalScore)
Scoreboard -> Game: sortScores()
Server -> Client: sendAction("showResults")
Client -> Player: displayScore()
@enduml
```

10. Appendix


«device»
Application Server
«device»
PC/Laptop
«execution environment»
Browser
«component»
Scrabble Game UI
«protocol»
Http/Https
«protocol»
WebSocket (ws:)
«execution environment»
Spring Boot Server
«component»
Game Controller
(StartGameView)
«component»
WebSocket Handler
(ScrabbleConsumer)
«protocol»
Internal API
«protocol»
Internal API
«storage»
Database
«library»
JPA/Hibernate
«protocol»
SQL (via ORM)

```
// 1. WebSocket Setup (Client-side JavaScript):

let socket = new WebSocket('ws://your-server-url/ws/scrabble/');
socket.onmessage = function(e) {
    let data = JSON.parse(e.data);
    handleGameUpdate(data);
};
function sendMove(move) {
    socket.send(JSON.stringify({'type': 'move', 'data': move}));
}

// 2. WebSocket Handler (Server-side Java with Spring):

@ServerEndpoint("/ws/scrabble/{gameId}")
public class ScrabbleConsumer {
    @OnOpen
    public void connect(Session session, @PathParam("gameId") String gameId) {
        session.getUserProperties().put("game_id", gameId);
        self.accept();
    }

    @OnMessage
    public void receive(Session session, String textData) {
        JSONObject data = new JSONObject(textData);
        if (data.getString("type").equals("move")) {
            updateGameState(data.getJSONObject("data"));
        }
    }

    public void updateGameState(JSONObject move) {
        Game game = Game.objects.get(id=self.game_id);
        game.applyMove(move);
        game.save();
        async_to_sync(self.channel_layer.group_send)(
            "game_" + self.game_id,
            {"type": "game_update", "message": {"move": move, "state": game.getState()}}
        );
    }
}
```

```
// 3. Database Models (Spring JPA):

@Entity
class Player {
    @Id @GeneratedValue private Long id;
    private String name;
    @ManyToOne private User user;
}

@Entity
class Game {
    @Id @GeneratedValue private Long id;
    @OneToMany private List<Player> players;
    @Column(columnDefinition = "TEXT") private String currentState;

    public void applyMove(JSONObject move) {
        // Logic to apply move and update current_state
        pass
    }
    public JSONObject getState() {
        return new JSONObject(self.currentState);
    }
}

// 4. HTTP API (Spring Controller):
@RestController
class StartGameView {
    @PostMapping("/api/start-game")
    public Map<String, Object> post(@RequestBody Map<String, Object> request) {
        players = [Player.objects.get(id=id) for id in request.POST.getlist('player_ids')];
        game = Game.objects.create();
        game.players.set(players);
        game.save();
        return JsonResponse({"game_id": game.id});
    }
}

// 5. URL Configuration (urls.java):
urlpatterns = [
    path('api/start-game/', StartGameView.as_view(), name='start_game'),
    path('ws/scrabble/<int:game_id>/', ScrabbleConsumer.as_asgi()),
]
```